

Clase 3

Del CSV al monitor

Construir los gráficos del monitor desde su CSV, y entender el flujo completo: de dónde salen los datos, qué cadena de scripts hay detrás, y qué hacer cuando sale un dato nuevo.

Germán Rosati - german.rosati@gmail.com

Fundar — grupo factor~data

Objetivo de la clase

Las clases 1 y 2 fueron sobre **el gráfico como objeto**. Esta es sobre **el flujo que lo produce**: qué archivo lo dibuja, de dónde salió el CSV que lee, qué cadena de scripts hay detrás, y qué tiene que hacer alguien de la provincia cuando sale un dato nuevo.

MANTENIMIENTO

Construir un gráfico del monitor desde su CSV.

COMUNICACIÓN

Saber qué advertencia va en el caption.

Es la clase donde los dos perfiles hacen cosas distintas pero acopladas — la práctica final los obliga a trabajar juntos.

Anatomía de un script de visualización

```
library(tidyverse)
source('style/fundar_monitor_theme.R') # 1. tema

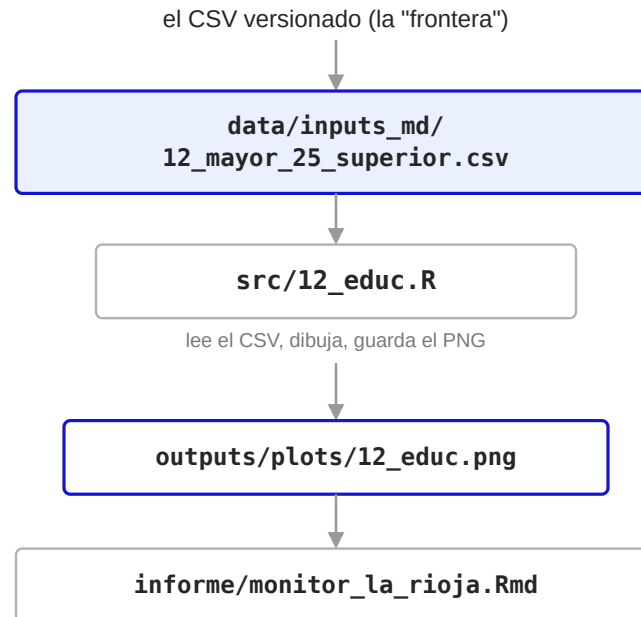
df <- read_csv('data/inputs_md/
  12_mayor_25_superior.csv')
# 2. LEE un CSV

df_plot %>%
  ggplot(aes(x = fecha, y = porc_...,
    color = la_rioja_region)) +
  geom_line() +
  ylim(0, 35) +
  scale_color_fundar_multi() +
  theme_monitor() + # 4. el gráfico
  labs(title = "...",
    caption = fuente_fundar(...))

ggsave('outputs/plots/12_educ.png')
# 5. ESCRIBE un PNG
```

- **Todos los scripts de viz son el mismo archivo.** Cambia el CSV, la columna y, el ylim y los textos. Si sabés hacer uno, sabés hacer los diez.
- **El script no calcula el indicador.** El número ya viene hecho en el CSV — solo lo dibuja. Por eso el CSV, y no el PNG, es lo que importa versionar.

La cadena completa



lo inserta con `mostrar("outputs/plots/12_educ.png")` — por eso el CSV, y no el PNG, es lo que importa versionar.

Recordatorio: la gramática de gráficos

En la clase 1 vimos que un gráfico se arma con un número chico de piezas que, combinadas, generan cualquier gráfico posible — la idea de Wilkinson de una **gramática**. En código, es `src/04_desoc.R` línea por línea:

```
df <- read_csv('./data/inputs_md/04_tasa_desoc.csv') # 1. DATOS

df_plot %>%
  ggplot(aes(x = fecha, y = tasa_desoc,                # 2. MAPEO ESTÉTICO
            group = la_rioja_region,
            color = la_rioja_region)) +
  geom_line(linewidth = 0.7) +                        # 3. OBJETO GEOMÉTRICO
  scale_color_fundar_multi(name = "Región") +         # 4. ESCALA (dato → color)
  ylim(0, 20) +                                       # 4. ESCALA (dato → posición y)
  theme_monitor() +                                  # 5. TODO LO QUE NO ES DATO
  labs(title = ..., x = ..., y = ..., caption = ...) # 6. ANOTACIÓN
```

ggplot2 se llama así por esto: "gg" es *grammar of graphics*. El + no es decorativo — es el operador que compone las capas de la gramática, y por eso los diez scripts de viz del repo son la misma receta con distintos ingredientes.

¿Dentro o fuera de aes()?

La distinción que separa un **mapeo** (depende del dato) de una **decisión constante** (no depende de nada):

DENTRO DE AES() → MAPEO

```
color = la_rioja_region
```

El color **depende de una columna** del dato: cada valor de `la_rioja_region` se traduce a un color distinto.

FUERA DE AES() → CONSTANTE

```
linewidth = 0.7
```

El grosor es **fijo**: no varía con ningún dato, es una decisión de diseño igual para todas las líneas.

Confundir estos dos es el error nº 1 de quien arranca con ggplot2. Va a volver en el bump chart: ahí el grosor de línea **sí** depende del dato (`linewidth = la_rioja_region`, dentro de `aes()`) — es una excepción deliberada, no un error.

Gráfico de líneas: lo que ya saben, y lo que lo vuelve del monitor

Aplicando la gramática que acabamos de repasar a un CSV nuevo. Las cuatro decisiones que separan un gráfico de exploración de uno **del monitor** —y viven casi todas en la capa de ESCALA— son:

DECISIÓN	EN EL CÓDIGO	POR QUÉ
factor(la_rioja_region)	Fuerza el orden 1<2<3	La Rioja se dibuja última: queda por encima cuando las líneas se cruzan.
Eje Y en cero	ylim(0, 35)	Sin esto, ggplot recorta y exagera la variación (integridad visual).
Paleta del proyecto	scale_color_fundar_multi()	Sin esta línea, tres colores de igual peso y La Rioja no salta.
Eje X domado	scale_x_discrete(breaks = quiebres_x)	fecha es texto, 75 categorías → el truco deja 2 marcas por año.

Alternativa más robusta: `lubridate::yq(fecha) + scale_x_date()` — la que se usa en la práctica.

Bump chart: la pregunta que responde

¿En qué puesto está La Rioja respecto de las otras provincias, y cómo cambió ese puesto? — algo que la serie de tiempo no responde bien: muestra el nivel, pero no si se está ganando o perdiendo posiciones.

QUÉ ES

También se lo llama **slopegraph de ranking**: una familia de gráfico de líneas donde el eje Y no es un valor, sino la **posición** de cada unidad en un ranking, año a año. El repo lo usa en `src/15_pbg_ranking_percapita.R` para el PBG per cápita.

Datos: `15_pbg_per_capita_por_provincia.csv` — 24 provincias × 15 años (2010–2024).

El script completo del bump chart

```
library(tidyverse)
source("style/fundar_monitor_theme.R")

prov <- read_csv("data/inputs_md/15_pbg_per_capita_por_provincia.csv") # 1. LEE

rank_df <- prov %>%
  group_by(anio) %>%
  mutate(ranking = min_rank(desc(pbg_per_capita))) %>% # 2. TRANSFORMA
  ungroup()

ggplot(rank_df, aes(anio, ranking, group = provincia, color = la_rioja_region)) +
  geom_line(aes(linewidth = la_rioja_region)) +
  geom_text(data = etiqueta_lr, aes(label = ranking)) +
  scale_y_reverse(breaks = 1:24) +
  scale_color_fundar_multi() +
  scale_linewidth_manual(values = c(0.3, 0.5, 1.4)) +
  theme_monitor() + # 3. DIBUJA
  labs(title = "...", caption = fuente_fundar(...))

ggsave("outputs/plots/clase3_bump_pbg.png") # 4. GUARDA
```

Mismo esqueleto de siempre: **lee** → **dibuja** → **guarda**. Lo único que se mete en el medio es un paso de **transformación** antes de graficar — el tema de la próxima slide.

Comparación: el script de líneas vs. el del bump

12_EDUC.R — LÍNEA

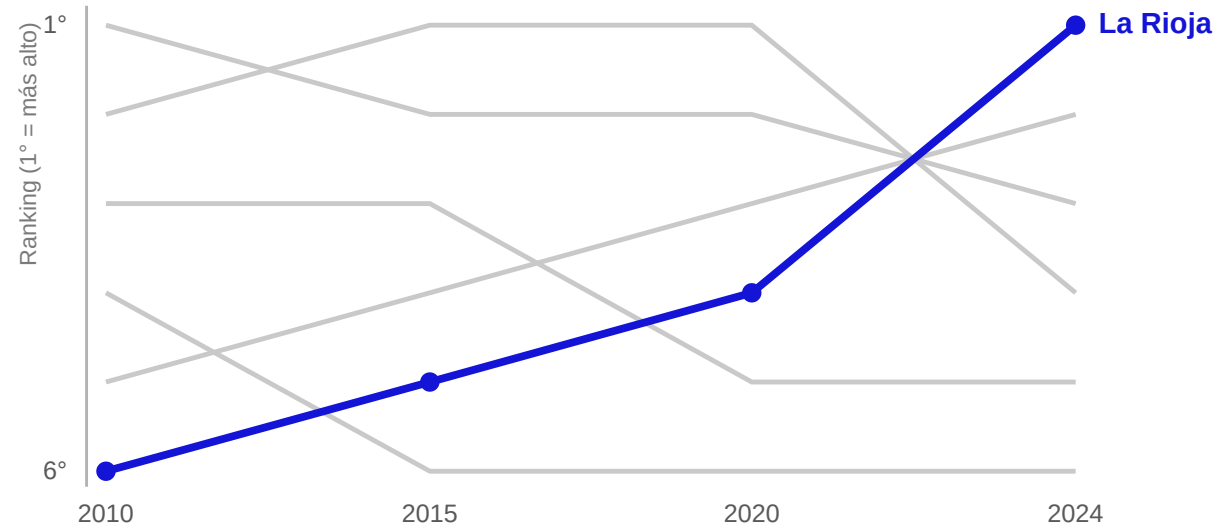
1. **LEE** `read_csv(csv)`
2. **DIBUJA** `geom_line() + scale_color_fundar_multi() + theme_monitor()`
3. **GUARDA** `ggsave(png)`

02_BUMP_PBG.R — BUMP

1. **LEE** `read_csv(csv)`
2. **TRANSFORMA** `(nuevo) mutate(ranking = min_rank(...))`
3. **DIBUJA** `geom_line() + geom_text() + scale_color_fundar_multi() + scale_y_reverse() + scale_linewidth_manual() + theme_monitor()`
4. **GUARDA** `ggsave(png)`

Comparten **LEE**, **GUARDA**, y hasta `scale_color_fundar_multi() + theme_monitor()` dentro de **DIBUJA**. El bump agrega un paso de **TRANSFORMA** antes de graficar, y dos capas de escala más (`scale_y_reverse()`, `scale_linewidth_manual()`) — nada que no sea la misma gramática, aplicada con dos ingredientes de más.

Así se ve un bump chart



Cada línea es una provincia. El eje Y no es un valor: es el **puesto** en el ranking de ese año, con el 1° arriba. En este ejemplo, La Rioja arranca última y termina primera — el bump chart deja ver esa trayectoria de un vistazo, cosa que una tabla de rankings anuales no logra.

Bump chart: la misma gramática, tres cambios

data → aes → geom → scale → theme → labs. Un bump chart **es** un gráfico de líneas — lo que cambia son tres decisiones. La primera:

CAMBIO 1 DE 3 · DE VALOR A RANKING

```
rank_df <- prov %>%  
  group_by(anio) %>%  
  mutate(ranking = min_rank(desc(pbg_per_capita))) %>%    # 1 = el más alto ese año  
  ungroup()
```

El gráfico **no dibuja pbg_per_capita**, dibuja ranking. Muchas veces el gráfico no grafica la columna que viene en el CSV, sino algo derivado de ella en el propio script de viz: acá es un ranking; en otros casos es un índice base 100, una media móvil, una variación interanual.

Por qué el ranking sí se puede comparar entre años aunque el nivel no. El PBG per cápita está en pesos constantes de 2004, así que el nivel ya es comparable — pero incluso si estuviera en pesos corrientes (como el salario del indicador 03), el **ranking dentro de un mismo año** sería inmune a la inflación, porque afecta a las 24 jurisdicciones por igual.

Cambio 2 de 3: el puesto 1 va arriba

```
ggplot(rank_df,  
       aes(anio, ranking,  
           group = provincia)) +  
  geom_line() +  
  scale_y_reverse(breaks = 1:24)
```

- Un ranking se lee con **el mejor arriba**. Sin `scale_y_reverse()`, ggplot dibuja el eje Y de manera creciente hacia arriba y el puesto 1 queda **abajo**.
- El resultado se lee al revés de como lo espera el ojo: subir de puesto se vería como "bajar" en el gráfico.

Pregunta para la sala: ¿qué pasa si sacamos `scale_y_reverse()`? — el gráfico se da vuelta y "subir en el ranking" se ve como bajar.

Cambio 3 de 3: el patrón figura/fondo

24 líneas del mismo color son ilegibles. La solución es la misma paleta regional del monitor: las provincias de contexto en un tono claro que no se puede —ni se necesita— seguir individualmente, y La Rioja destacada.

-  **Contexto** (21 provincias) — línea fina, tono claro.
-  **La Rioja** — línea gruesa, la única que se sigue punto por punto.

```
ggplot(rank_df, aes(anio, ranking, group = provincia, color = la_rioja_region)) +  
  geom_line(aes(linewidth = la_rioja_region)) +  
  scale_color_fundar_multi() +  
  scale_linewidth_manual(values = c(0.3, 0.6, 1.4))  # contexto fino, La Rioja gruesa
```

Como el CSV ya trae `la_rioja_region`, el mapeo sale directo. Reemplaza a la leyenda una **etiqueta al final de la línea** de La Rioja (`geom_text` sobre el último año, con `coord_cartesian(clip = "off")`).

Bump chart: preguntas para la sala

¿POR QUÉ GROUP = PROVINCIA Y NO GROUP = LA_RIOJA_REGION?

Porque hay 24 líneas, una por provincia; el color agrupa de a tres pero la línea es por provincia.

¿QUÉ PASA SI SACO SCALE_Y_REVERSE() ?

El gráfico se da vuelta y "subir en el ranking" se ve como bajar.

NO PUEDO DISTINGUIR LAS 18 PROVINCIAS DEL "RESTO PAÍS" ENTRE SÍ. ¿ES UN PROBLEMA?

No: son contexto. La pregunta del gráfico es sobre La Rioja, no sobre el resto.

*El gráfico es soporte de una narrativa. Si el texto habla de cuatro cosas, no muestres doce. **La granularidad del gráfico tiene que igualar la del texto.***

Informe final Argendata, págs. 40–42

El gráfico es soporte de un texto

Caso ENGHo: el análisis publicado menciona cuatro rubros de consumo, el gráfico desagrega doce. La solución no es un gráfico más lindo: es **desagregar solo lo que el texto discute** y agregar el resto.

En el monitor pasa lo mismo con las cinco sub-dimensiones de NBI: hoy 13a_nbi_hogares .R grafica solo el total, y está bien, porque el análisis habla del total.

Los tres campos de texto de un gráfico

CAMPO	QUÉ TIENE QUE DECIR	EJEMPLO
Título	El hallazgo, afirmado	"La desocupación en La Rioja se ubica por debajo del promedio del NOA desde 2021"
Subtítulo	Unidad, universo, período, transformación	"Desocupados como % de la PEA. Aglomerado La Rioja, 2007–2025, promedio de 4 ondas"
Caption	Fuente + advertencias metodológicas	"Serie interrumpida entre 2015-T3 y 2016-T1"

MEJOR EJEMPLO

03b_salarios_registrados_EPH.R: el caption marca el cambio de metodología de ingresos (2015-2016) y **el gráfico dibuja esa línea.**

PEOR EJEMPLO

04_desoc.R: el título es el nombre de la variable, no dice nada. Los diez indicadores tienen el mismo problema.

La Rioja es un dominio chico de la EPH

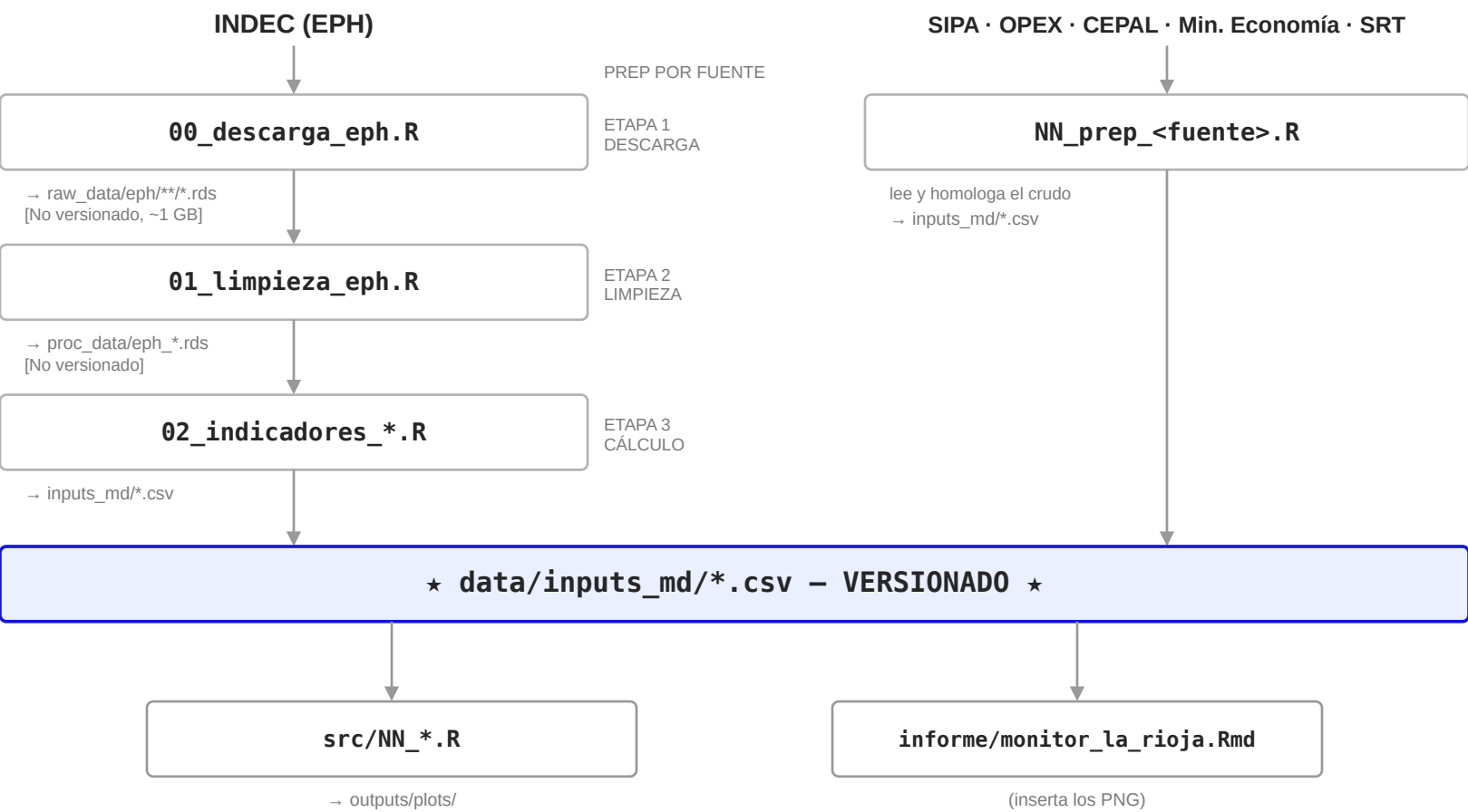
- Nunca reportar un nivel a partir de **un solo trimestre**: promediar las cuatro ondas del año.
- Un 0,00 en una sub-dimensión de NBI **no significa "no hay privación"** — significa que la muestra no alcanza para medirlo.
- En el bump: un cambio de **un solo puesto** entre años consecutivos es probablemente ruido, no tendencia.

Si se publica, esta advertencia va en el caption.

**La lógica del monitor:
de dónde sale cada número**

Del dato crudo al gráfico publicado

La EPH (Encuesta Permanente de Hogares, INDEC) es la fuente de la mitad de los indicadores del monitor. No es la única fuente, pero es la única con **tres** etapas en vez de una.



Zoom: las tres etapas de la EPH, script por script

ETAPA 1 · DESCARGA

00_descarga_eph.R

- Descarga incremental de microdatos trimestrales con el paquete eph.
- Si el .rds del trimestre ya existe, lo saltea.

→ data/raw_data/eph/**/* .rds
no versionado, ~1 GB

ETAPA 2 · LIMPIEZA

01_limpieza_eph.R

- Canoniza tipos y nombres de columnas.
- Deriva variables (ej. mayor_25_superior).
- ⚠ el orden de `as.character()` importa (trampa 2).

→ data/proc_data/eph_*.rds
no versionado

ETAPA 3 · CÁLCULO

02_indicadores_eph_*.R

- Un bloque de código por indicador.
- Pondera por PONDERA / PONDII0 según el indicador.

→ data/inputs_md/*.csv
★ versionado ★ — lo que consumen los gráficos

Por qué tres scripts y no uno: para poder re-ejecutar solo una parte. La descarga completa tarda varios minutos; recalculer un indicador sobre datos ya limpios, segundos.

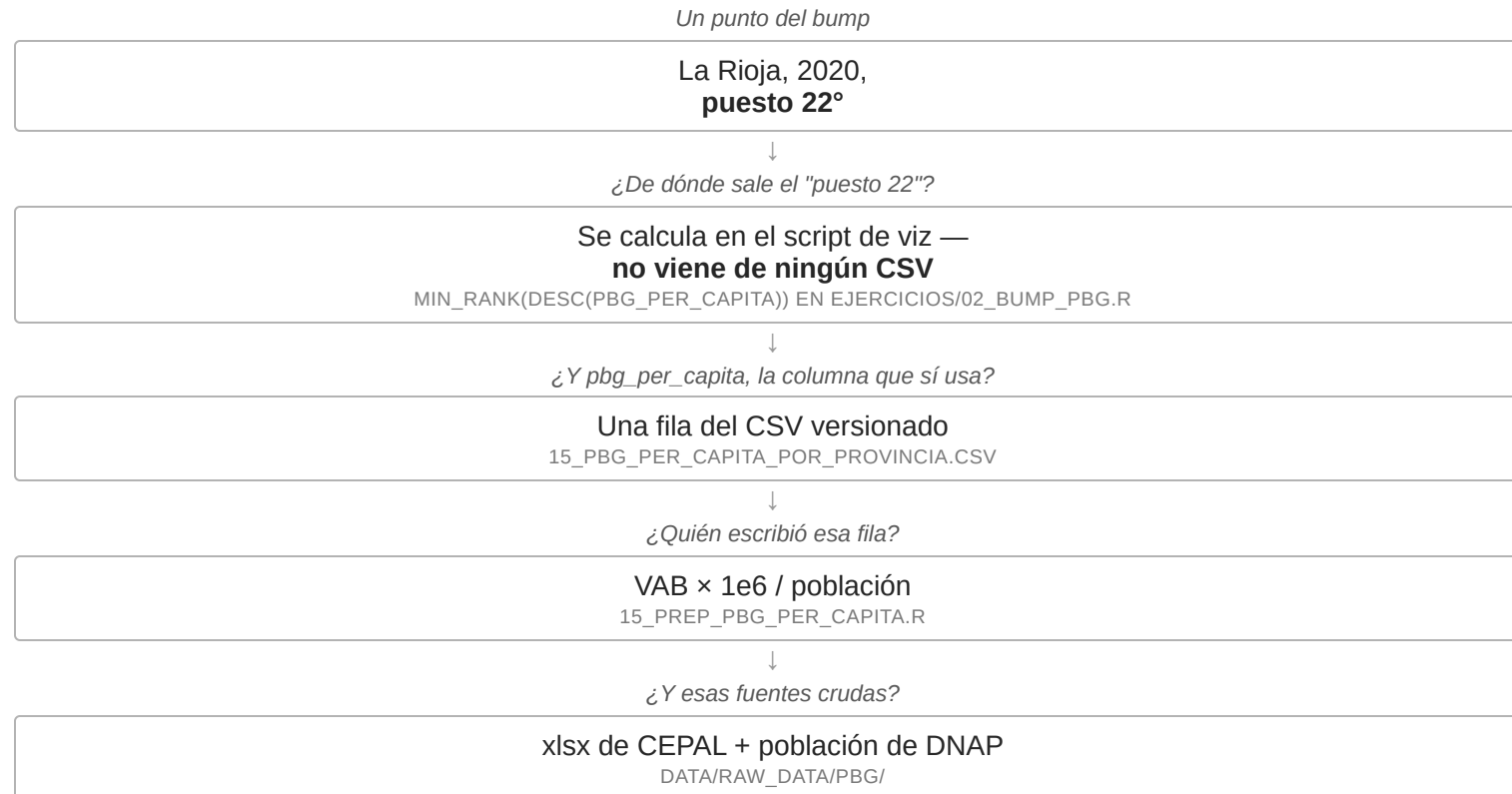
Decisiones tomadas para el diseño del flujo

1. **Por qué la EPH tiene tres etapas y no un script:** recalculer un indicador no requiere volver a descargar ni relimpiar 19 años de microdatos. Las fuentes no-EPH son más chicas y colapsan a un solo prep.
2. **Por qué raw_data/ no se versiona** pero inputs_md/ sí: los microdatos pesan ~1 GB y se regeneran solos; los CSV agregados pesan poco y se revisan en un diff — si un indicador cambia, se ve exactamente en qué.
3. **La línea con la estrella es la frontera del taller.** Arriba: equipo de datos. Abajo: equipo de comunicación. El contrato es el CSV: si tiene las columnas esperadas y los datos correctos, las dos mitades trabajan en paralelo sin pisarse.

Recorrido inverso: de un punto del gráfico al código



Recorrido inverso, más corto: el bump



A diferencia del indicador de educación, el primer paso hacia atrás acá **no busca un CSV**: se recalcula cada vez que corre el script (la *Comparación* de la slide 10). La EPH tampoco aparece en esta cadena — el PBG es una fuente distinta, con su propio prep.

Las cuatro trampas

- **La descarga incremental no re-descarga.** Si agregás una variable, los `.rds` existentes no se actualizan — hay que borrarlos y volver a correr `00`.
- **El orden de `as.character()` en la limpieza.** Al revés, las comparaciones de texto dejan de matchear *en silencio, sin error*. El bug más difícil de detectar del pipeline.
- **El quiebre 2015/2016.** `PONDII0` no existe antes de 2016 — los niveles de salarios a ambos lados no son comparables.
- **La Rioja es un dominio chico.** Sub-dimensiones de `NBI` en `0,00` por muestra insuficiente, no por ausencia de privación.

Las trampas 3 y 4 son exactamente lo que tiene que aparecer en el caption de un gráfico publicado.

Cómo se actualiza

SALIÓ DATO NUEVO DE...	PASOS	FRECUENCIA
EPH continua	00_descarga_eph.R → 01 → 02_* → viz	Trimestral
SIPA	Reemplazar .xlsx → NN_prep_* → viz	Mensual
CEPAL / PBG	Reemplazar Excel → 15_prep_pbg.R → 15_prep_pbg_per_capita.R → viz	Cuando publiquen
OPEX, SRT, finanzas, educación	Reemplazar raw → prep → viz	Anual

Fuerza bruta cuando no se sabe qué se tocó: `source("src/999_run_pipeline.R")`.

El circuito de publicación

```
git checkout -b actualizo-eph-2026q2 # NUNCA sobre main
git add data/inputs_md/ outputs/plots/
git commit -m "Actualizo indicadores EPH, onda 2026-T2"
git push -u origin actualizo-eph-2026q2
# → abrir un Pull Request en GitHub
```

El push a main publica. El workflow de GitHub Actions renderiza el sitio y lo publica en gh-pages — **actualizar un CSV dispara esa republicación sola**. La rama y el PR permiten que otro mire el cambio antes de que salga.

Práctica integradora — en parejas de perfiles mezclados

El foco es leer, no escribir: nada de código para completar en clase — eso queda como refuerzo opcional para la casa.

1 • LECTURA DE GRÁFICOS

Antes/después del gráfico de líneas, y errores de integridad plantados en el de desocupación — se identifican sin ver el código.

2 • PREDICCIÓN

Cuatro variantes del código del bump: predecir qué cambia en el gráfico antes de leer la respuesta.

Consigna completa en `practica.md`.

El mapa de pendientes del repositorio

- **Títulos que nombran variables** en vez de afirmar hallazgos — reescribir los diez `labs()`.
- `scale_x_date()` en todos los scripts de viz — media hora de trabajo, diez archivos.
- **El indicador 03 en pesos corrientes** — falta deflactar con IPC (el pendiente de mayor impacto).
- **Integrar los indicadores 17 y 18** al `informe/monitor_la_rioja.Rmd`.

1. Un gráfico es un mapeo de datos a propiedades visuales — los canales no son intercambiables.

2. La pregunta define la herramienta, y un gráfico claro igual puede mentir.

3. El circuito es reproducible de punta a punta: el CSV es el contrato entre los dos equipos.